Shadowing 2.0

Reading, Writing e Speaking Challenge

Como cada feature vai funcionar, onde ela entra no app, quais ferramentas usar e o que a equipe precisa decidir

> **Objetivo da reunião:** ler este material em conjunto e **sair com as perguntas Q1 a Q11 decididas** (seção 7). Cada pergunta traz as opções, o resultado esperado de cada uma, os trade-offs e uma recomendação. A linha "Decisão da equipe" fica em branco para preencher na reunião.

Roteiro sugerido (60 min)

Tempo	Bloco	Seções
5 min	Ponto de partida e o novo layout do app	1 e 2
15 min	As três features, passo a passo	3, 4 e 5
5 min	O que é compartilhado e quanto custa	6
30 min	Perguntas e decisões	7
5 min	Quadro de decisões e próximos passos	8

*Reunião de 07/11/2026 · Base: branch **Membro4**, commit **e48fa44** · Material de apoio, não versionado. Preços e limites de LLM pesquisados em novembro de 2026: eles mudam com frequência, confira no painel de cada provedor antes de contratar.*

1. Ponto de partida: o que já existe e serve para as três features

O Shadowing já tem a base que as features novas precisam. Ninguém precisa recriar estas peças, só reaproveitar. Os guias em `docs/guias/` mostram o caminho de cada uma.

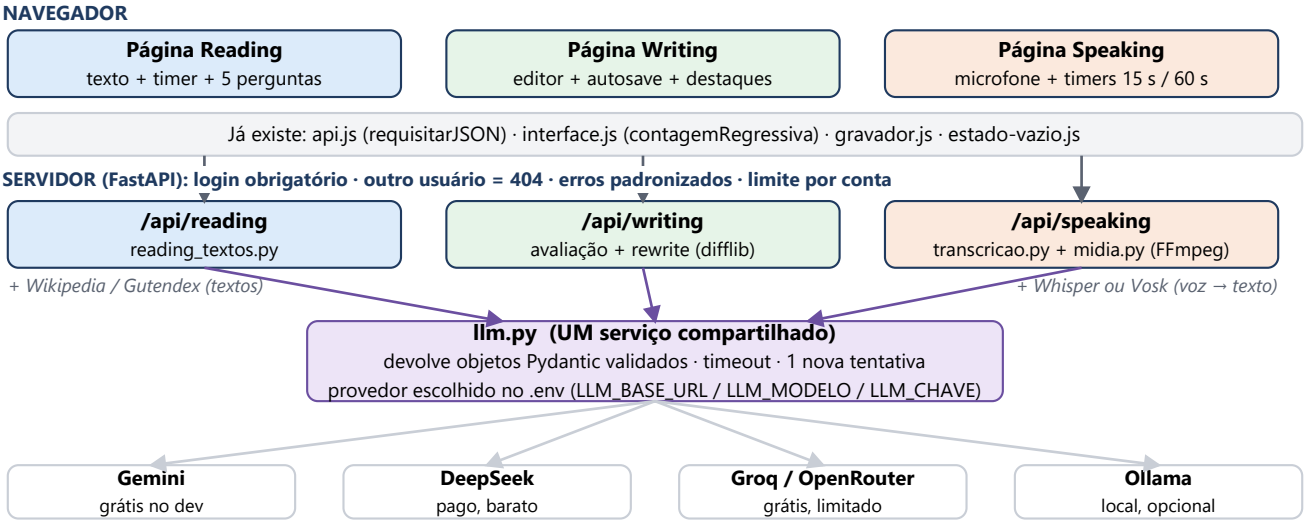
Já existe	Onde	Serve para
Contas, login e sessões; toda rota /api exige login	app/api/auth.py, test_protecao_api.py	Reading, Writing, Speaking
Isolamento: recurso de outro usuário responde 404	padrão _deck() em app/api/decks.py	Todas (desafios são por usuário)
Formato padrão de erro {detail, codigo}	app/erros.py (ErroDaApi)	Falha da LLM, timeout, áudio inválido
Limitador de uso por conta (429)	app/api/legendas.py (B11)	Controlar o custo da LLM
Timer de contagem regressiva	static/js/nucleo/interface.js	Tempo de leitura, redação, 15 s / 60 s
Salvar com atraso + salvar ao sair	static/js/nucleo/configuracoes.js	Autosave do Writing
Gravação do microfone (MediaRecorder)	static/js/componentes/gravador.js	Speaking
Upload de áudio + conversão FFmpeg	app/api/avaliacoes.py, app/services/midia.py	Speaking
Reconhecimento de voz offline (Vosk)	app/services/alinhamento.py	Speaking (alternativa ao Whisper)
Cliente da Wikimedia (User-Agent, timeout)	app/services/dicionario.py	Reading (busca de textos)
Migrações Alembic (banco na 0006)	migrations/, python -m app.migrar	Tabelas novas de cada feature
Suíte de testes (224 SQLite / 225 Postgres / 13 E2E)	tests/	Base para os testes novos

> **Ponto-chave:** as três features usam LLM. A escolha do provedor **não trava ninguém**: nos testes a LLM é sempre substituída por uma versão falsa, e o provedor real é só configuração no `.env`. Cada membro pode começar já.

2. O novo layout do app

2.1 Arquitetura: como as três features se encaixam

Cada feature tem a própria página, a própria rota e as próprias tabelas, mas todas passam pelo mesmo núcleo (login, erros, limite) e pelo **mesmo serviço de LLM**. Trocar o provedor muda uma linha do `.env`, não o código das features.



Camada	Reading (Membro 1)	Writing (Membro 3)	Speaking (Membro 2)
Página / JS	paginas/reading.js	paginas/writing.js	paginas/speaking.js
Rotas	/api/reading/...	/api/writing/...	/api/speaking/...
Serviço próprio	busca e limpeza de textos	avaliação, destaques, rewrite	transcrição + métricas de fala
Usa a LLM para	gerar 5 perguntas, corrigir	corrigir redação, julgar rewrite	avaliar a fala transcrita
Migrações	0101, 0102, ...	0301, 0302, ...	0201, 0202, ...
Peso no servidor	baixo (rede + LLM)	baixo (só LLM)	**alto** (voz → texto)

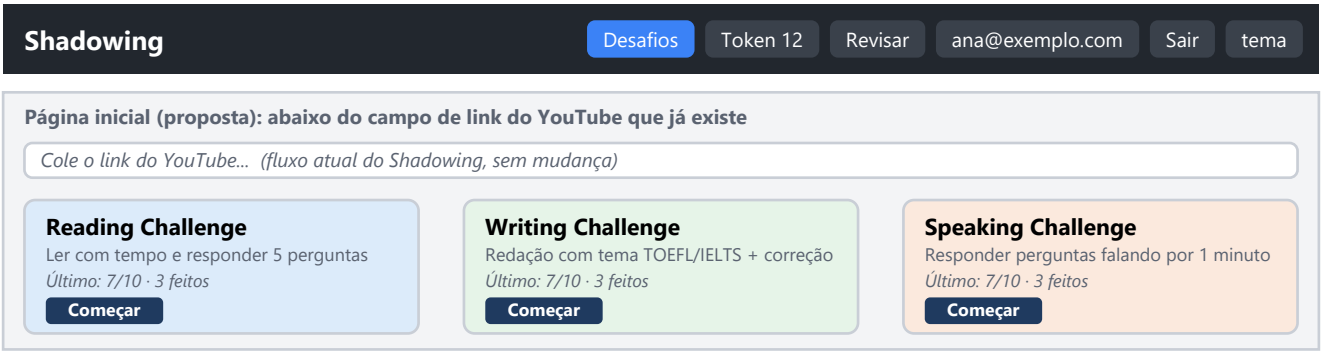
2.2 Onde as features aparecem na tela

Hoje a barra superior tem: Token, Revisar, e-mail, Sair e tema. A proposta abaixo é um ponto de partida para a pergunta **Q9**; as páginas novas usam o mesmo `base.html`, o mesmo tema claro/escuro e a mesma navegação HTMX das páginas atuais.

Hoje



Proposta: um menu "Desafios" (abre Reading / Writing / Speaking)

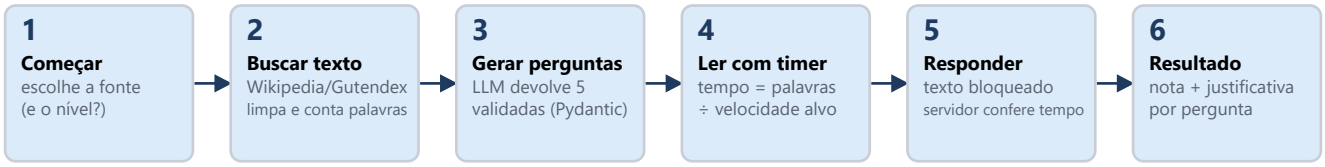


Todas as páginas de desafio seguem o mesmo esqueleto, o que facilita para o usuário e para o código:

- **Tela inicial do desafio:** botão "Começar" + histórico das tentativas (com o componente de tela vazia que já existe).
- **Tela em andamento:** timer no topo, conteúdo no centro, ação principal embaixo (Enviar / Próxima).
- **Tela de resultado:** nota geral, notas por critério, comentários e "Tentar de novo".
- **Estados de espera e erro:** "processando..." e mensagens de erro no formato padrão, com botão para tentar novamente.

3. Reading Challenge (Membro 1)

O que o usuário faz: lê um texto em inglês com tempo limitado e depois responde cinco perguntas de compreensão, sem poder voltar ao texto.



Como funciona por dentro

- **Texto:** a Wikipedia entrega o texto já sem HTML (`prop=extracts&explaintext=1`). Funções puras removem referências, seções vazias e listas, e descartam textos curtos ou impróprios.
- **Perguntas:** uma chamada à LLM com o texto pede **exatamente 5** perguntas em JSON. Se vier inválido, tenta de novo uma vez e depois usa um fallback. As perguntas ficam gravadas: recarregar a página não gera outras.
- **Tempo:** o horário de início é gravado no banco. O bloqueio vale também no servidor (o timer da tela pode ser burlado).
- **Correção:** múltipla escolha é corrigida pelo próprio servidor, sem LLM; respostas abertas precisam de uma segunda chamada à LLM (ver **Q8**).

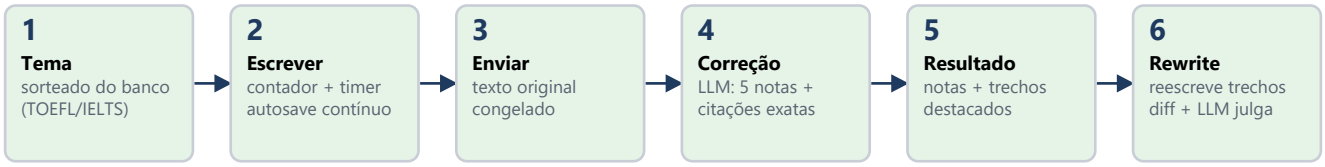
Dados e rotas

Tabela	Guarda	Dono
ReadingText	texto limpo, fonte, nº de palavras	compartilhado (como a waveform)
ReadingChallenge	texto escolhido, início, tempo limite	usuário
ReadingQuestion	enunciado, alternativas (JSON), gabarito	via desafio
ReadingAttempt	respostas, nota, justificativas	usuário

Pontos de atenção	Por quê
Artigos aleatórios ruins (listas, muito curtos, temas sensíveis)	Precisa de filtro e de uma nova tentativa de busca
Obras do Gutenberg em inglês antigo	Podem ficar difíceis demais; talvez limitar a autores/épocas (Q7)
Custo: 1 a 2 chamadas à LLM por desafio	Limite por conta (Q10)
Wikipedia/LLM fora do ar	Erro 502 no formato padrão; o usuário tenta de novo

4. Writing Challenge e Rewrite (Membro 3)

O que o usuário faz: escreve uma redação sobre um tema no estilo TOEFL/IELTS com tempo limitado, recebe uma correção por categoria com os trechos problemáticos destacados e depois reescreve esses trechos.



Como funciona por dentro

- **Autosave:** o texto vai para o servidor alguns segundos depois que a pessoa para de digitar, a cada N segundos e ao sair da página ("Salvo às 14:32"). Sessão vencida ou queda de internet não perdem a redação.
- **Estados:** rascunho → enviada → avaliada (ou erro_avaliacao). **A redação é salva antes de chamar a LLM**: se a correção falhar, aparece "Avaliar de novo" e o texto continua lá.
- **Correção:** a LLM devolve nota e comentário para gramática, vocabulário, coerência, organização e atendimento à tarefa, mais a **citação exata** de cada trecho problemático. O servidor localiza a citação no texto (LLMs erram posições) e descarta as que não existem.
- **Segurança:** o texto foi digitado pelo usuário; os destaques são montados sem `innerHTML` (evita XSS).
- **Rewrite:** cada reescrita é uma linha nova, nunca sobrescreve o original. O `difflib` do Python mostra o que mudou e a LLM julga se melhorou. O histórico original → feedback → reescrita fica completo.

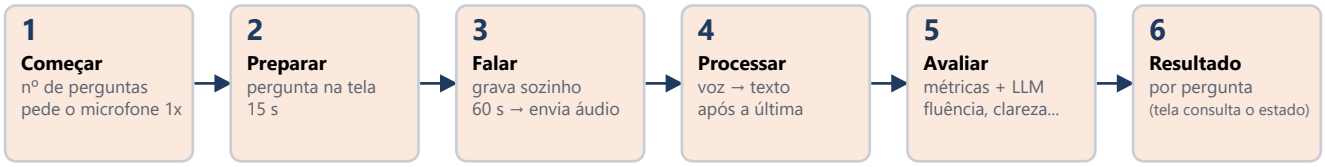
Dados

Tabela	Guarda	Dono
WritingPrompt	temas (carregados por migração de dados)	compartilhado
WritingAttempt	texto, estado, início, nº de palavras	usuário
WritingFeedback	notas (JSON), trechos, modelo e versão do prompt	via tentativa
WritingRewrite	trecho reescrito, avaliação da melhora	via feedback

É a feature mais leve: só texto, sem áudio e sem fonte externa além da LLM. O ponto mais caro é a correção de redações longas (limite de tamanho do texto no servidor).

5. Speaking Challenge / One Minute Talk (Membro 2)

O que o usuário faz: responde uma bateria de perguntas falando. Para cada pergunta tem 15 s para se preparar e 60 s para responder; a gravação começa e termina sozinha. No fim, recebe métricas da fala e uma avaliação.



Como funciona por dentro

- **Microfone:** o navegador só pede permissão a partir de um clique. O pedido acontece em "Começar bateria" e o microfone fica aberto, para a gravação começar sozinha depois dos 15 s.
- **Áudio:** Chrome grava webm e Firefox grava ogg; o servidor converte tudo com o FFmpeg (já existe). Limite de tamanho (413) e conferência da duração no servidor.
- **Processamento só depois da bateria:** cada upload só grava o arquivo. Na última resposta a sessão vai para "processando", o servidor transcreve fora da requisição e a tela consulta o estado até "concluída" (ver **Q6**).
- **Voz → texto:** a parte mais pesada do app inteiro. Opções na **Q5** (Whisper, Vosk, navegador ou API).
- **Métricas sem LLM** (a partir dos tempos de cada palavra): duração, palavras por minuto, pausas, % do minuto usado.
- **LLM:** uma chamada avalia as transcrições da bateria. Se ela falhar, a transcrição e as métricas já estão salvas.

Dados

Tabela	Guarda	Dono
SpeakingSession	estado (em_andamento → processando → concluida / erro), nº de perguntas	usuário
SpeakingQuestion	enunciado (banco fixo ou gerado pela LLM)	compartilhado ou via sessão
SpeakingAnswer	transcrição, métricas (JSON), avaliação; o áudio NÃO vai para o banco	via sessão

Atenção: é a feature que mais demora e mais muda o Docker e o servidor de produção (biblioteca nativa, modelo de centenas de MB, CPU e memória). As decisões Q5 e Q6 definem quase todo o risco dela.

6. O que é compartilhado e quanto custa

6.1 Um único serviço de LLM para as três features

DeepSeek, Gemini, Groq, OpenRouter e Ollama aceitam o formato de API da OpenAI. Um só cliente (`openai` do Python) atende todos, trocando três variáveis:

```
LLM_BASE_URL=https://generativelanguage.googleapis.com/v1beta/openai/  # Gemini
# LLM_BASE_URL=https://api.deepseek.com                                  # DeepSeek
# LLM_BASE_URL=http://127.0.0.1:11434/v1                                # Ollama local
LLM_MODELO=    LLM_CHAVE=    LLM_TIMEOUT_SEGUNDOS=60    # sem valores reais no Git
```

- Funções devolvem objetos Pydantic já validados; JSON inválido → 1 nova tentativa → erro padrão.
- Sem chave configurada o app sobe normalmente; só as features de LLM respondem com erro claro.
- Limite de uso por conta com o limitador que já existe (um "espaço" por feature).
- Códigos de erro novos (ex.: llm_tempo_esgotado, llm_indisponivel) entram em CODIGOS, no README e nos testes.

6.2 Quanto de LLM cada desafio usa (estimativa)

Feature	Chamadas por desafio	Entrada	Saída
Reading	2 (gerar perguntas; corrigir abertas)	~2.700 tokens	~1.100 tokens
Writing	1 a 2 (corrigir; julgar o rewrite)	~2.500 tokens	~1.300 tokens
Speaking	1 (avaliar as transcrições)	~1.500 tokens	~700 tokens

1 token ≈ 0,75 palavra em inglês. Modelos com "raciocínio estendido" podem multiplicar a saída por 2 a 5: para estas tarefas, usar modos sem raciocínio estendido.

6.3 Opções gratuitas e pagas (nov/2026)

Provedor	Grátis?	Limite / preço	Observação
Gemini Flash	Sim, sem cartão	~5-15 req/min, ~1.000-1.500 req/dia	Boa qualidade; limites mudam; no grátis o Google pode usar os dados
Groq	Sim	~30 req/min; 70B ~1.000 req/dia	Muito rápido; limite de tokens/min baixo
OpenRouter :free	Sim	20 req/min, 50 req/dia (1.000 após US$ 10)	Fica lento no pico; falhas contam no limite
DeepSeek V4 Flash	Não (muito barato)	US$ 0,30 entrada / 1,20 saída por 1M (pico)	~US$ 0,003 por desafio; servidores na China
DeepSeek V4 Pro	Não	US$ 1,32 / 3,96 por 1M (pico)	~US$ 0,01 por desafio
Ollama (local)	Sim, sem limite	custo é o hardware (8 GB+ de RAM livre)	Lento sem GPU; qualidade menor (modelos 7-8B)

Cenário (DeepSeek V4 Flash)	Desafios por mês	Custo estimado
Desenvolvimento e testes manuais	~500	~US$ 1,50
Turma de 20 pessoas, 3 desafios/dia	~1.800	~US$ 5
100 usuários ativos, 3 desafios/dia	~9.000	~US$ 25

O gasto só foge do controle sem limite de uso. Quase todo provedor permite um teto de gasto mensal no painel.

7. Perguntas para decidir

Para cada pergunta: as opções, o resultado esperado, os trade-offs e a recomendação. A recomendação é um ponto de partida para a discussão, não uma decisão.

Q1. Qual LLM usar durante o desenvolvimento?

Afeta todos os membros já nas próximas semanas.

	Opção	Resultado esperado	Prós	Contras / trade-offs
A	**Gemini Flash (grátis)**	Todos desenvolvem sem custo e sem cartão.	Boa qualidade e JSON com schema	Limites mudam; 5-15 req/min; dados podem ser usados pelo Google
B	**Groq (grátis)**	Respostas muito rápidas.	Velocidade; modelos abertos	Limite de tokens/min estoura com redações longas; modelos menores
C	**OpenRouter :free**	Acesso a vários modelos com um cadastro.	Flexível	50 req/dia é pouco para 3 pessoas testando
D	**Ollama local em cada máquina**	Sem limite e offline.	Grátis e sem limite	8 GB+ de RAM, lento, qualidade e comportamento diferentes da produção

Recomendação: A, com D como opção para quem tiver máquina boa ou ficar sem cota.	**Decisão da equipe:** _____

Q2. Qual LLM usar na apresentação e no deploy?

Define custo, risco no dia da apresentação e privacidade.

	Opção	Resultado esperado	Prós	Contras / trade-offs
A	**DeepSeek V4 Flash (pago)**	~US$ 5/mês para uma turma.	Muito barato; boa qualidade	Precisa de crédito pré-pago; dados vão para a China; preços mudaram várias vezes em 2026
B	**Gemini Flash (pago)**	Mesma faixa de custo.	Mesmo modelo do desenvolvimento: zero surpresa	Precisa de cartão no Google Cloud
C	**Gemini Flash (grátis)**	Custo zero.	Sem gasto	Risco real de erro 429 (limite) no meio da apresentação
D	**Ollama no servidor**	Sem custo por token.	Dados não saem do servidor	Servidor com muita RAM/GPU custa mais que a API; lento

Recomendação: B (ou A se o custo pesar), sempre com teto de gasto no painel.	**Decisão da equipe:** _____

Q3. O usuário final deve rodar uma LLM local (Ollama) no próprio computador?

Ideia levantada para evitar custo de API.

	Opção	Resultado esperado	Prós	Contras / trade-offs
A	**Não: só API no servidor**	Funciona para qualquer usuário, em qualquer máquina.	Simples; correção confiável no servidor	Tem custo de API (pequeno, ver 6.3)
B	**Opcional (avançado)**	Quem quiser roda o app localmente apontando para o Ollama.	Útil para devs e uso offline	Duas configurações para documentar
C	**Obrigatória**	Cada usuário instala o Ollama e baixa ~5 GB.	Custo zero de API	No deploy o servidor não alcança o PC do usuário: a correção iria para o navegador (nota forjável, CORS, prompts expostos); 1-3 min por correção sem GPU; suporte caso a caso

Recomendação: A para usuários; B só como ferramenta da equipe.	**Decisão da equipe:** _____

Q4. Quem cria o serviço compartilhado llm.py, e quando?

Sem isso, surgem três integrações diferentes com a LLM.

	Opção	Resultado esperado	Prós	Contras / trade-offs
A	**Um membro cria a base antes**	Os três começam usando a mesma interface.	Sem retrabalho; um padrão só	Uma pessoa precisa fazer isso primeiro
B	**O primeiro colega que precisar**	Os outros adaptam depois.	Não espera ninguém	Interface pensada para uma feature só; ajustes na integração
C	**Cada um faz o seu**	Três clientes de LLM.	Total independência	Três configurações, três tratamentos de erro, mais difícil de trocar de provedor

Recomendação: A ou B, combinando a interface (nomes das funções e do .env) hoje.	**Decisão da equipe:** _____

Q5. Speaking: como transformar a voz em texto?

A decisão de maior impacto em tempo, Docker e servidor.

	Opção	Resultado esperado	Prós	Contras / trade-offs
A	**faster-whisper no servidor**	Melhor transcrição, com pontuação e tempos por palavra.	Qualidade alta; funciona em qualquer navegador	Biblioteca nativa, modelo de 150 MB a 1,5 GB, CPU/memória no servidor, mais tempo de implementação
B	**Vosk (já está no projeto)**	Reaproveita o que o karaokê e a avaliação já usam.	Já instalado e no Docker; leve; tempos por palavra	Transcrição pior em fala livre; sem pontuação
C	**Reconhecimento do navegador (Web Speech)**	Transcreve enquanto a pessoa fala.	Zero custo de servidor	Só Chrome/Edge; áudio vai para o Google; sem tempos por palavra confiáveis
D	**API de transcrição na nuvem**	Envia o áudio para um provedor.	Sem peso no servidor; boa qualidade	Custo por minuto; áudio do usuário vai para terceiros; outra chave

Recomendação: A com modelo pequeno (base.en, int8); B como plano de contingência se o prazo apertar.	**Decisão da equipe:** _____

Q6. Speaking: como processar as respostas depois da bateria?

O requisito pede processamento só após o término.

	Opção	Resultado esperado	Prós	Contras / trade-offs
A	**Thread no próprio servidor + a tela consulta o estado**	"Processando..." até concluir.	Simples; sem infraestrutura nova; padrão já usado no download	Se o servidor reiniciar no meio, precisa reprocessar (estado "erro")
B	**Fila externa (Celery/RQ + Redis)**	Processamento em outro processo.	Robusto; escala	Mais um serviço no Docker e no deploy; mais tempo

Recomendação: A. B só se houver muitos usuários simultâneos.	**Decisão da equipe:** _____

Q7. Reading: de onde vêm os textos?

Define a qualidade e o nível dos textos.

	Opção	Resultado esperado	Prós	Contras / trade-offs
A	**Wikipedia aleatória**	Variedade infinita.	Grátis; texto já sem HTML	Artigos ruins (listas, curtos) exigem filtros
B	**Gutenberg (Gutendex)**	Literatura em domínio público.	Textos de qualidade	Inglês antigo, difícil; precisa recortar trechos
C	**A + B, usuário escolhe**	Duas fontes com a mesma interface.	Mais opções	Mais código e testes
D	**Textos selecionados no banco**	A equipe escolhe os textos.	Controle total de nível e conteúdo	Trabalho manual; pouca variedade

Recomendação: A com filtros, e D como reserva se a busca falhar. **Decisão da equipe:** _____

Q8. Reading: perguntas de múltipla escolha ou abertas?

Muda o custo e o tempo da correção.

	Opção	Resultado esperado	Prós	Contras / trade-offs
A	**Múltipla escolha**	Correção instantânea pelo servidor.	Sem 2ª chamada à LLM; nota objetiva	Avalia menos a compreensão profunda
B	**Abertas**	A LLM corrige cada resposta.	Avaliação mais rica	Mais custo e espera; nota mais subjetiva
C	**Misto (ex.: 4 + 1)**	Combina os dois.	Equilíbrio	Os dois fluxos para implementar e testar

Recomendação: A no começo; C depois, se der tempo. **Decisão da equipe:** _____

Q9. Onde as features aparecem no layout?

Ver os desenhos da seção 2.2.

	Opção	Resultado esperado	Prós	Contras / trade-offs
A	**Menu "Desafios" na barra superior**	Acesso de qualquer página.	Navbar continua enxuta	Um clique a mais
B	**Três botões na barra**	Acesso direto.	Mais rápido	Barra lotada no celular
C	**Cards na página inicial**	Espaço para descrição e histórico.	Mais claro para quem chega	Só aparecem na página inicial

Recomendação: A + C (menu para navegar, cards para apresentar). **Decisão da equipe:** _____

Q10. Limite de uso e privacidade

Toda chamada à LLM custa, e os textos dos usuários saem do servidor.

	Opção	Resultado esperado	Prós	Contras / trade-offs
A	**Limite diário por feature + aviso**	Ex.: 10 desafios/dia; aviso de que o texto vai para um provedor de IA.	Custo previsível; transparência	Precisa escolher os números
B	**Só o teto de gasto do provedor**	Sem limite no app.	Nada para implementar	Um usuário pode esgotar a cota de todos
C	**Sem limite**	—	—	Risco de custo e de abuso

Recomendação: A, reaproveitando o limitador do B11. **Decisão da equipe:** _____

Q11. Integração do código e das migrações

Três branches criando tabelas ao mesmo tempo.

	Opção	Resultado esperado	Prós	Contras / trade-offs
A	**Faixas 01xx/02xx/03xx + alembic merge no final**	Cada um trabalha sem esperar o outro.	Já descrito nos guias; sem bloqueio	Um passo de merge na integração (Membro 4)
B	**Ordem combinada; cada um atualiza a branch antes**	Migrações em sequência.	Histórico linear	Um membro espera o outro; conflitos frequentes

Recomendação: A (já está nos guias).	**Decisão da equipe:** _____

8. Quadro de decisões e próximos passos

	Pergunta	Decisão	Responsável	Prazo
Q1	Qual LLM usar durante o desenvolvimento?			
Q2	Qual LLM usar na apresentação e no deploy?			
Q3	O usuário final deve rodar uma LLM local (Ollama) no próprio computador?			
Q4	Quem cria o serviço compartilhado llm.py, e quando?			
Q5	Speaking: como transformar a voz em texto?			
Q6	Speaking: como processar as respostas depois da bateria?			
Q7	Reading: de onde vêm os textos?			
Q8	Reading: perguntas de múltipla escolha ou abertas?			
Q9	Onde as features aparecem no layout?			
Q10	Limite de uso e privacidade			
Q11	Integração do código e das migrações			

Próximos passos sugeridos

- Criar as chaves do provedor escolhido na Q1 (cada membro a sua, **nunca** no Git).
- Combinar hoje a interface do llm.py (Q4): nomes das funções, variáveis do .env e códigos de erro.
- Cada membro segue o próprio guia em `docs/guias/` e cria as migrações na sua faixa.
- Membro 2: fazer primeiro um teste rápido da opção escolhida na Q5 (transcrever um áudio de 60 s e medir tempo, CPU e memória).
- Cada um anota para o Membro 4 as bibliotecas novas (com versão), as variáveis do .env e as URLs externas, para o Docker e o deploy (B14-B16).

Fontes dos preços e limites: páginas e guias de preço do DeepSeek (BenchLM, Maximem, CloudZero, NxCode), Gemini (Klymentiev, TinkerLLM, CloudZero, CostGoat), Groq (TokenMix, LocalAIMaster, eesel), OpenRouter (central de ajuda oficial, set/2026) e documentação do Ollama sobre saídas estruturadas. As fontes divergem em vários números: confira no painel oficial de cada provedor.